

Description du code `adrsinsta.py`

Le code implémente une simulation numérique en 1D pour une équation d'advection–diffusion–réaction–source (ADRS).

1. Équation considérée

L'équation aux dérivées partielles résolue est :

$$u_t = -Vu_x + Ku_{xx} - \lambda u + f(x, t),$$

où :

- V est la vitesse d'advection,
- K est le coefficient de diffusion,
- λ est le paramètre de réaction,
- $f(x, t)$ est le terme source (construit dans la fonction `fex`).

2. Solution exacte fabriquée (méthode de la solution manufacturée)

La fonction `fex(NX, dx, time)` calcule :

$$T_{\text{ex}}(x, t), \quad \partial_t T_{\text{ex}}(x, t), \quad \partial_x T_{\text{ex}}(x, t), \quad \partial_{xx} T_{\text{ex}}(x, t).$$

Le terme source $f(x, t)$ est alors construit de telle manière que T_{ex} satisfasse exactement l'équation ADRS. On parle ici de *solution manufacturée*.

3. Méthode numérique

- **Discrétisation spatiale** : différences finies centrées pour u_x et u_{xx} .
- **Discrétisation temporelle** : schéma de type Runge–Kutta multi-étages (coefficients α_k).
- **Pas de temps** : Δt choisi avec une condition CFL de stabilité, dépendant de dx , V , K , et de $\max(F)$.

4. Boucle temporelle

À chaque pas de temps, on effectue :

1. Calcul de $f(x, t)$ et de T_{ex} via `fex`.
2. Mise à jour de la solution numérique T avec l'équation discrétisée.
3. Évaluation du résidu (erreur de mise à jour) et ajustement de Δt .
4. Calcul de l'erreur quadratique $\|T - T_{\text{ex}}\|_{L^2}$ ainsi qu'une erreur de type H^1 (via les dérivées).

5. Études de convergence

Le code effectue plusieurs simulations avec un nombre croissant de points NX (boucle de raffinement de maillage).

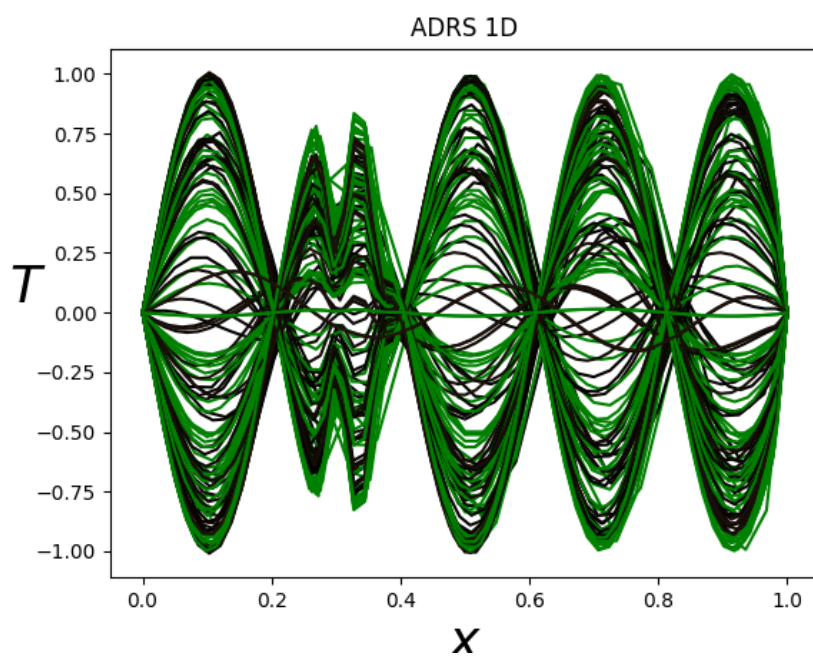
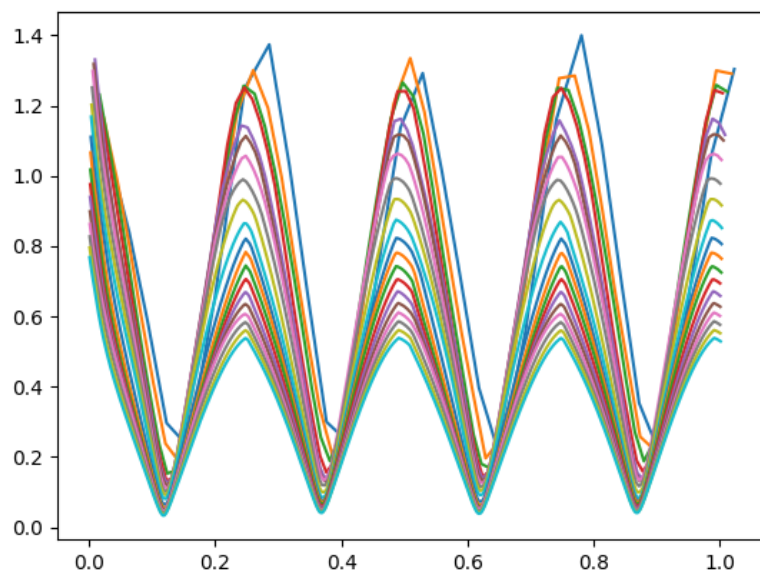
Il trace ensuite :

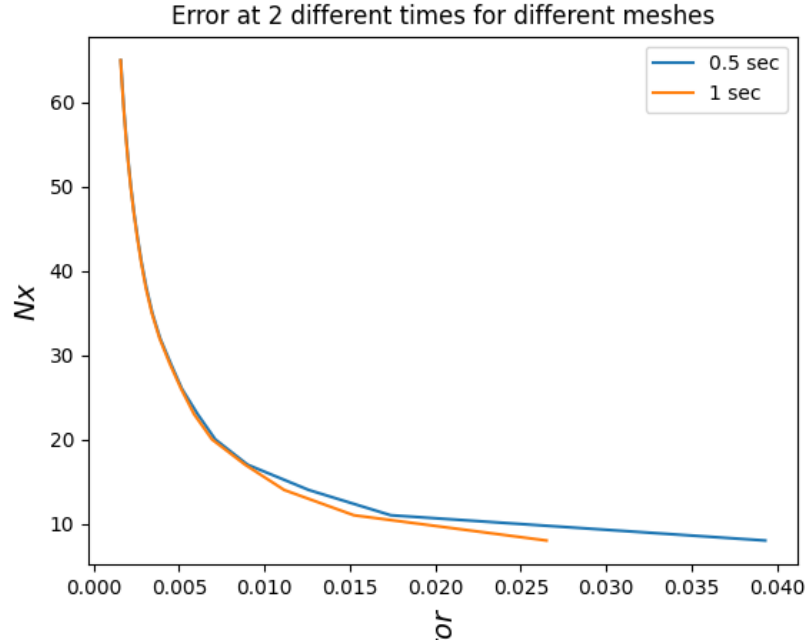
- l'évolution temporelle de la solution numérique $T(x)$ comparée à $T_{\text{ex}}(x)$,
- la décroissance du résidu en fonction du temps,
- l'évolution de l'erreur en norme L^2 en fonction du nombre de points NX , pour comparer la précision lorsque le maillage est raffiné.

6. Résultats produits

- **Figure 1** : solution numérique $T(x)$ et solution exacte $T_{\text{ex}}(x)$ à différents instants.
- **Figure 2** : résidu en fonction du temps (mesure de convergence).
- **Figure 3** : erreur en norme L^2 aux temps $t = 0.5$ et $t = 1$ en fonction du nombre de points NX .

Résumé : ce code met en œuvre un solveur numérique 1D par différences finies pour l'équation ADRS, vérifie la méthode par une solution manufacturée, et étudie la convergence en fonction du raffinement spatial.





Modifications du code et analyse des erreurs

L'objectif des modifications était double :

1. Visualiser l'erreur en norme L^2 à $T/2$ et à T pour différents maillages uniformes.
2. Visualiser l'évolution de l'erreur au point milieu du domaine pour différents schémas de Runge-Kutta (RK1 à RK4).

1. Étude de convergence spatiale

Pour étudier l'effet du raffinement spatial, nous avons :

- Introduit une boucle sur le nombre de points de grille NX afin de simuler des maillages de plus en plus fins.
- Calculé l'erreur en norme L^2 à chaque pas de temps :

$$\text{err} = \frac{\sqrt{\sum_j (T_j - T_{\text{ex},j})^2 dx}}{NX}.$$

- Enregistré spécifiquement l'erreur aux instants $T/2$ et T pour chaque maillage :

Err_half et Err_final.

- Tracé ces erreurs en fonction de NX , ce qui permet de visualiser la convergence spatiale.

2. Étude de l'ordre de Runge-Kutta

Pour étudier l'effet de l'ordre du schéma Runge-Kutta, nous avons :

- Fixé un maillage uniforme ($NX = 50$) pour toutes les simulations afin d'avoir une comparaison cohérente.
- Bouclé sur les ordres RK1 à RK4 et calculé les coefficients α correspondant à l'ordre :

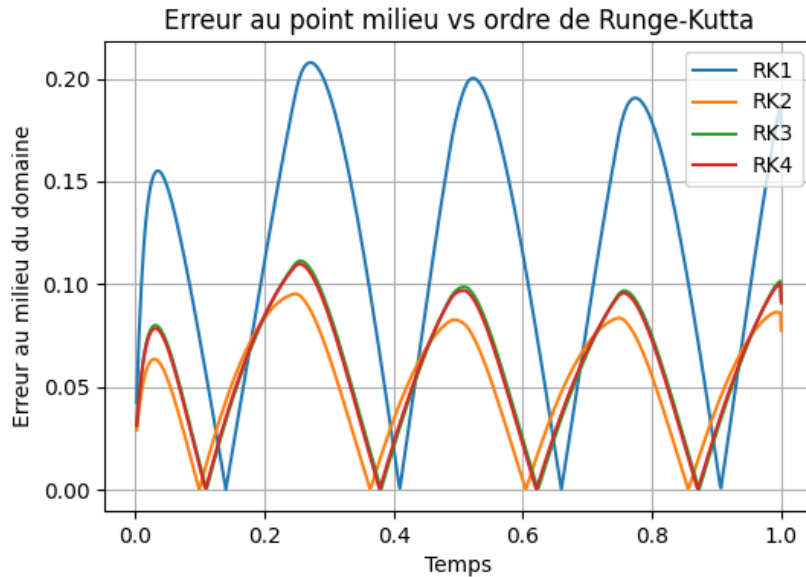
$$\alpha_k = \frac{1}{\text{ordre RK} - k}, \quad k = 0, \dots, \text{ordre RK} - 1.$$

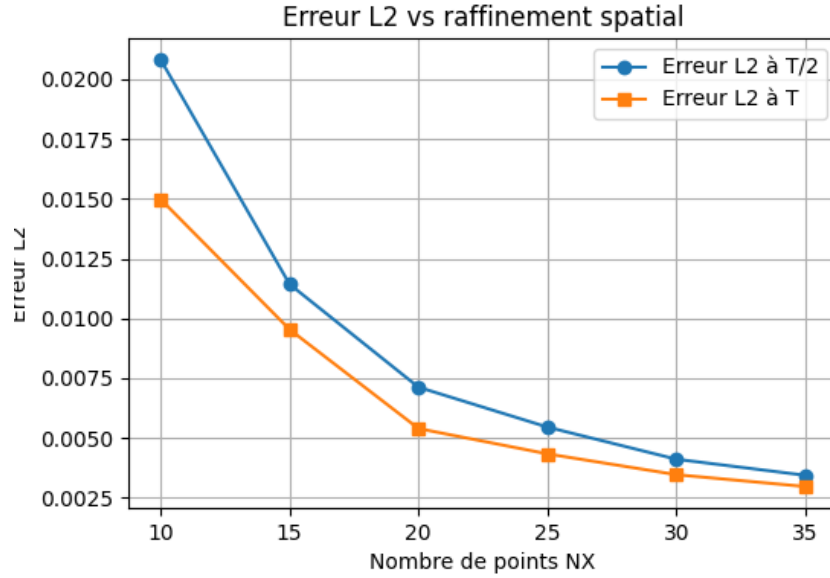
- À chaque pas de temps, enregistré l'erreur absolue au point milieu du domaine :

$$\text{err_mid}(t) = |T_{\text{num}}(x_{\text{mid}}, t) - T_{\text{ex}}(x_{\text{mid}}, t)|.$$

- Tracé l'évolution temporelle de cette erreur pour chaque ordre RK, permettant de comparer la précision locale de chaque schéma.

Ainsi, ces modifications permettent d'analyser à la fois la convergence spatiale du schéma et l'effet de l'ordre du Runge-Kutta sur la précision locale de la solution numérique.





Code `adrs.insta.multiple.mesh.adapt.py`

Le code Python étudié implémente une simulation numérique d'un problème 1D d'advection-diffusion-réaction avec source. L'équation résolue est de la forme :

$$\frac{\partial u}{\partial t} + V \frac{\partial u}{\partial x} - K \frac{\partial^2 u}{\partial x^2} + \lambda u = \text{source}(x, t) \quad (1)$$

où :

- V est la vitesse d'advection,
- K est le coefficient de diffusion,
- λ est le coefficient de réaction,
- $\text{source}(x, t)$ est un terme forcé pour suivre une solution test u_{ex} .

L'objectif principal du code est de simuler l'évolution temporelle de $u(x, t)$ tout en adaptant localement le maillage pour capturer les variations importantes de la solution.

Paramètres physiques et numériques

Paramètres physiques

- $K = 0.01$ (diffusion)
- $V = 1.0$ (advection)

- $\lambda = 1.0$ (réaction)
- $\text{freq} = 7$ (fréquence du terme source)
- Domaine spatial : $x \in [0, 1]$
- Temps total de simulation : $T = 2$

Paramètres numériques

- Nombre initial de points de grille : $NX = 3$
- Nombre maximal de pas de temps : $NT = 10000$
- Pas de grille minimal et maximal pour adaptation : $h_{\min} = 0.01$, $h_{\max} = 0.5$
- Critère d'erreur pour adaptation : $\text{err} = 0.01$

Méthode numérique

Discretisation spatiale

- L'espace est discrétisé sur un maillage 1D, initialement uniforme, puis ****adaptatif****. - La dérivée première est calculée par différences finies centrées :

$$\frac{\partial u}{\partial x} \approx \frac{u_{j+1} - u_{j-1}}{x_{j+1} - x_{j-1}}$$

- La dérivée seconde (diffusion) :

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{\frac{u_{j+1} - u_j}{x_{j+1} - x_j} - \frac{u_j - u_{j-1}}{x_j - x_{j-1}}}{0.5(x_{j+1} + x_j - x_j - x_{j-1})}$$

Schéma temporel

- Avancement explicite :

$$T_j^{n+1} = T_j^n + \Delta t \left(-V \frac{\partial T}{\partial x} + (K + \nu_{\text{num}}) \frac{\partial^2 T}{\partial x^2} - \lambda T_j + \text{source}_j \right)$$

- ν_{num} est une ****viscosité numérique**** pour stabiliser l'advection.

Conditions aux limites

- $T_0 = 0$ (Dirichlet)
- $T_{NX-1} = 2T_{NX-2} - T_{NX-3}$ (Neumann second ordre)

Adaptation de maillage

- La grille est raffinée là où la solution varie rapidement, grâce à une métrique basée sur la seconde dérivée :

$$\text{metric}_j = \max \left(\frac{1}{h_{\max}^2}, \min \left(\frac{1}{h_{\min}^2}, \frac{|T_{xx}|}{\text{err}} \right) \right)$$

- Le pas local devient :

$$h_{\text{loc}} = \sqrt{\frac{1}{\text{metric}}}$$

- Interpolation linéaire utilisée pour transférer la solution sur le nouveau maillage.

1 Résultats

- La solution $T(x, t)$ est tracée après chaque itération de maillage adaptatif.
- Les graphiques montrent comment la solution suit le profil gaussien imposé par le terme source.
- Le maillage est plus fin au centre du domaine où la solution varie rapidement, et plus grossier aux extrémités.

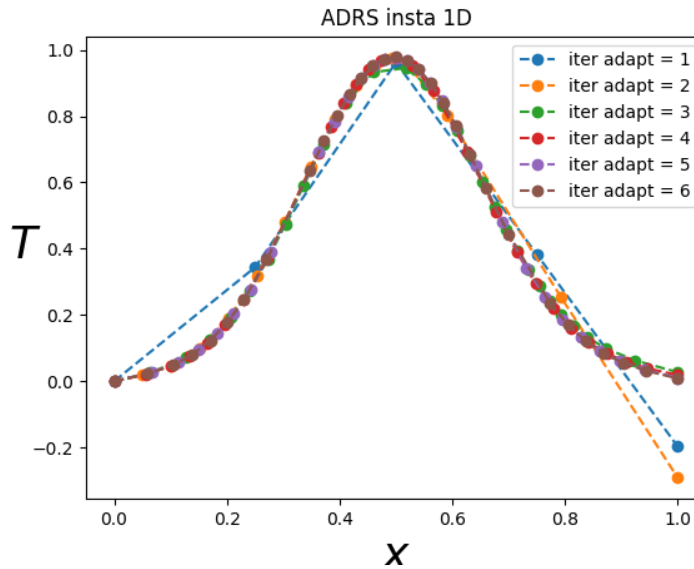


Figure 1: Exemple de solution $T(x, t)$ après adaptation de maillage.

L'objectif principal du code est de simuler l'évolution temporelle de $u(x, t)$ tout en adaptant localement le maillage pour capturer les variations importantes de la solution.

2 Codage de ce code à partir du code `adrs.multiple.mesh.adap.py`

Problème étudié dans notre premier code

On considère l'équation d'advection–diffusion–réaction–source (ADRS) en 1D :

$$u_t = -Vu_x + Ku_{xx} - \lambda u + f(x, t), \quad (2)$$

où V est la vitesse d'advection, K le coefficient de diffusion, λ le coefficient de réaction et $f(x, t)$ le terme source.

Solution exacte imposée

Afin de tester la méthode numérique, nous choisissons une solution exacte de la forme séparée :

$$u_{\text{ex}}(t, x) = u(t) v(x), \quad (3)$$

avec

$$u(t) = \sin(4\pi t), \quad v(x) = \exp(-20(x - 0.5)^2). \quad (4)$$

Il s'agit d'une oscillation temporelle sinusoïdale modulée spatialement par un profil gaussien.

Construction du terme source

En remplaçant u_{ex} dans l'équation ADRS, on obtient le terme source

$$f(x, t) = u'(t)v(x) + Vu(t)v_x(x) - Ku(t)v_{xx}(x) + \lambda u(t)v(x), \quad (5)$$

où

$$u'(t) = 4\pi \cos(4\pi t). \quad (6)$$

Les dérivées spatiales v_x et v_{xx} sont approchées numériquement par différences finies.

Méthode numérique

La discrétisation repose sur un schéma **Euler explicite en temps** et **différences finies centrées en espace** :

$$T^{n+1} = T^n + \Delta t \left(-VT_x + KT_{xx} - \lambda T + f(x, t) \right). \quad (7)$$

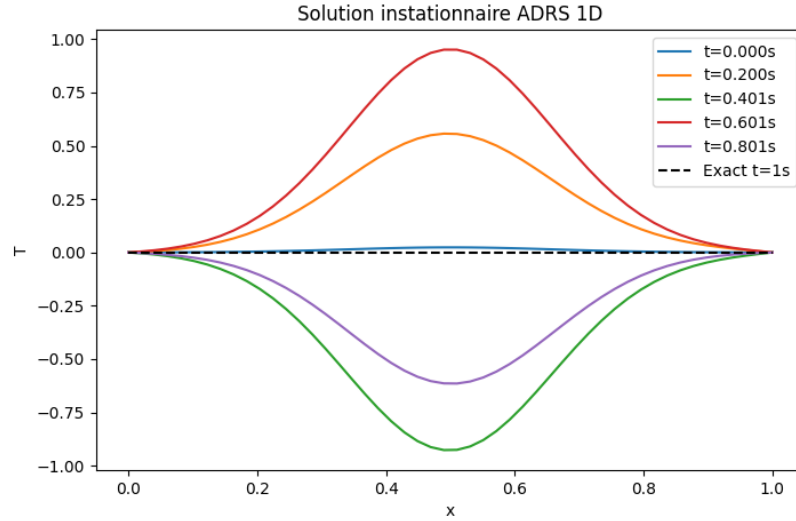
Le pas de temps Δt est choisi selon un critère CFL afin d'assurer la stabilité.

Caractère instationnaire

Contrairement au cas stationnaire traité précédemment, le résidu ne converge pas vers zéro car la solution est instationnaire et oscille dans le temps. Nous faisons donc évoluer la solution jusqu'au temps final fixé $T = 1$ s, sans critère d'arrêt sur le résidu.

Résultats numériques

La solution est représentée à différents instants entre $t = 0$ et $t = 1$ s, puis comparée à la solution exacte au temps final. On observe que la solution numérique reproduit correctement la dynamique temporelle imposée par la source et reste proche de la solution analytique, sous réserve de respecter la condition CFL.



Critères d'arrêt pour l'adaptation

L'adaptation de maillage repose sur une boucle : résolution $\rightarrow$ estimation de l'erreur $\rightarrow$ adaptation. Afin de contrôler à la fois la précision et le coût numérique, nous introduisons un critère mixte d'arrêt:

$$\|u_h - u_{\text{ex}}\|_{L^2} \leq \varepsilon, \quad (8)$$

$$N \leq N_{\text{max}}. \quad (9)$$

L'itération d'adaptation n'est arrêtée que si **les deux conditions sont satisfaites simultanément**. Cela empêche un arrêt prématuré (erreur trop grande) et un surraffinement inutile (nombre de points trop élevé).

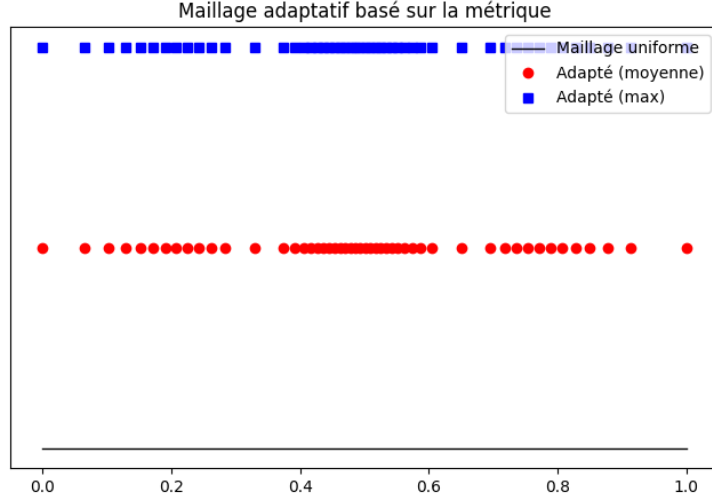
Maillage adaptatif stationnaire

Le maillage peut être adapté à partir d'une métrique construite sur la solution finale $u(x, T)$. En pratique, on définit une métrique proportionnelle à la courbure de la solution :

$$\mathcal{M}(x) \sim |u_{xx}(x, T)|. \quad (10)$$

Ce procédé conduit à un raffinement localisé dans les zones où la solution finale présente de fortes variations spatiales. Toutefois, dans notre cas particulier, la solution finale $u(x, T) = \sin(4\pi T)v(x)$

est nulle (car $\sin(4\pi T) = 0$). La métrique stationnaire est alors uniforme, et le maillage ne reflète pas la structure réelle du problème. Il est donc préférable de définir la métrique à partir du profil spatial $v(x)$, ou bien d'une enveloppe temporelle.



Adaptation en instationnaire

Pour mieux capturer l'évolution de la solution dans le temps, on introduit une *métrique globale* basée sur les solutions intermédiaires. À chaque instant t_n , on définit une métrique locale :

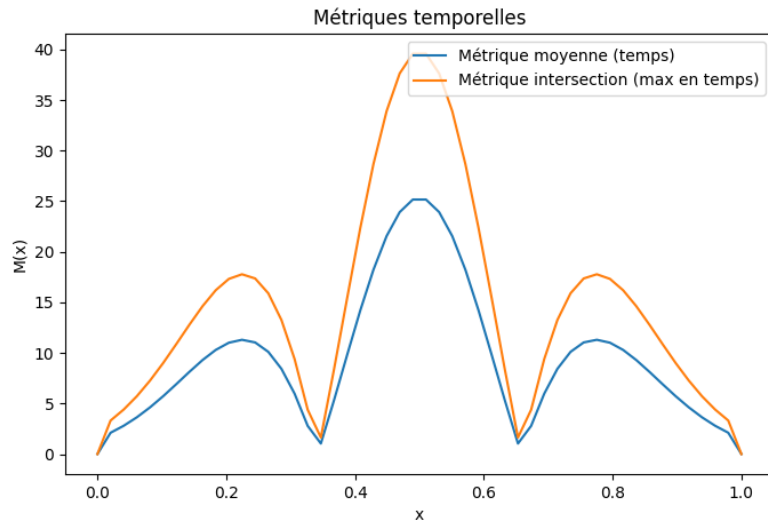
$$\mathcal{M}_n(x) \sim |u_{xx}(x, t_n)|. \quad (11)$$

On construit ensuite une métrique globale en temps par :

$$\mathcal{M}(x) = \frac{1}{N_t} \sum_{n=1}^{N_t} \mathcal{M}_n(x) \quad (\text{moyenne temporelle}), \quad (12)$$

ou

$$\mathcal{M}(x) = \max_{1 \leq n \leq N_t} \mathcal{M}_n(x) \quad (\text{intersection temporelle}). \quad (13)$$



Cette approche permet d'obtenir un maillage adapté à l'ensemble de l'évolution temporelle de la solution, garantissant que les structures transitoires sont correctement résolues.